

ANLP Assignment 1: Cipher Transformer Ablation Study

Parth Gupta

September 2026

1 Overview

The task was to build an encoder-decoder Transformer that decrypts a substitution cipher. Each plaintext sentence (128 characters) is encoded as a 1024-bit binary string (8 bits per character). The dataset has 5,946 pairs split 80/10/10. We compare five configurations (C1 to C5), each changing exactly one component from the base model.

2 Hyperparameters per model

All models share: $d_{\text{model}}=256$, $d_{\text{ff}}=1024$, 4 encoder + 4 decoder layers, 8 heads, batch size 64, Adam with $\text{lr } 3 \times 10^{-4}$, 2000-step warmup, label smoothing 0.1, dropout 0.1, 50 epochs with patience of 5.

3 All models

3.1 C1 - Base Transformer (Sinusoidal PE + MHA + LayerNorm + BPE)

Standard Seq2Seq Transformer with a BPE tokenizer implemented from scratch of vocab size (1,000) for both source (cipher) and target (plaintext). This is the base model upon which all other modifications will be made. Source and target sequences are capped at 512 tokens.

3.2 C2 - Rotary Positional Embeddings (RoPE)

Sinusoidal PE is replaced with RoPE. We applied complex-valued rotations to the query and key vectors in every attention layer. The rotation frequencies follow $\theta_i = 10000^{-2i/d}$. RoPE is applied after the Q/K linear projections and before the dot-product attention, operating on pairs of dimensions. The same RoPE cache is shared across encoder and decoder self-attention; cross-attention uses RoPE only on the query side.

3.3 C3 - Grouped Query Attention (GQA)

MHA is replaced with GQA using 2 KV heads shared across 8 query heads (4:1 grouping). The key and value projections output $2 \times (d_{\text{model}}/8) = 64$ dimensions total instead of $8 \times 32 = 256$. Each KV head is repeated to match its group of 4 query heads before the scaled dot-product. This saves $\sim 15\%$ of total parameters.

3.4 C4 - RMSNorm

LayerNorm is replaced with RMSNorm in all encoder and decoder layers. RMSNorm normalizes by $\text{RMS}(x) = \sqrt{\frac{1}{d} \sum x_i^2}$ without subtracting the mean, and uses a single learnable scale γ (no bias). This removes one learnable vector per normalization layer.

3.5 C5 - Byte Latent Transformer (BLT)

Token free architecture that doesn't use BPE like other models. Key implementation choices based on the information either taken from the paper or later clarified through mail:

1. **Bit-to-byte conversion:** The 1024-character binary string is parsed into 128 actual bytes by grouping every 8 bits. We don't treat individual ASCII '0'/'1' characters as bytes.
2. **Entropy-based dynamic patching:** A 3-gram Markov model is fit on the training cipher bytes. At inference time, we compute the Shannon entropy of the next byte predictive distribution at each position. A patch boundary is placed wherever entropy exceeds $\tau=2.0$. This produces variable-length patches rather than fixed width chunks.
3. **Local encoder:** Within each dynamic patch, a 2-layer local Transformer ($d_{\text{local}}=128$, 4 heads) runs self-attention restricted to bytes in the same patch (enforced via a cumsum based boolean mask). A learnable query then pools each patch into a single d_{model} -dimensional vector via cross-attention.
4. **Global encoder decoder:** The pooled patch vectors pass through the same 4+4 layer global Transformer as C1–C4. The decoder operates on raw target bytes (vocab size 259: 256 byte values + PAD + BOS + EOS).

4 Improving Accuracy: From Naive to Final

The initial naive implementation achieved only $\sim 70\%$ bit accuracy. Several targeted changes compounded to reach 88 to 99% accuracy:

4.1 Vocabulary Size: 8,000 $\rightarrow$ 1,000

The single biggest accuracy boost for C1–C4 came from reducing the BPE vocab from 8,000 to 1,000.

- **Problem with large vocab:** At 8,000 merges, BPE learnt to compress entire common English words (e.g., “the”, “and”) into single tokens. The model then memorizes the *bit pattern* for each word as an atomic lookup, but fails on any word not in the training vocabulary-it never learns the underlying *per-character* cipher rule.

- **Why 1,000 worked:** At 1,000 merges, tokens correspond roughly to 2–3 character fragments. The model is forced to learn how individual 8-bit cipher bytes map to plaintext characters rather than memorizing whole-word patterns. This generalized far better to the test set.

4.2 Bit-to-Byte Conversion for C5

For BLT, converting the 1024 ASCII characters to 128 true bytes was the biggest improvement. The original implementation had treated each ‘0’/‘1’ character as a separate byte, which was both slow and wrong, the model would have had no signal that groups of 8 would form a coherent unit. After conversion, the sequence length dropped $8\times$ to just 128, aligning the input representation with the cipher’s actual structure and enabling near perfect accuracy.

4.3 Other Improvements

- **Label smoothing (0.1):** Prevented overconfident predictions and improved generalization.
- **Gradient clipping (1.0):** Stabilized training, especially for C5 where byte-level loss landscapes are sharper.
- **Dynamic patching vs. fixed patches:** Switching from fixed 9-byte patches to entropy-based dynamic patches improved C5’s ability to group semantically related bytes.

5 Results

5.1 Performance Comparison

Table 1: Test-set performance. Best values in **bold**. BLEU/ROUGE not applicable to C5 (byte-level output).

Config	Bit Acc (%)	Seq Acc (%)	Lev (Norm)	BLEU	Val Loss
C1 (Base)	88.2	21.8	0.031	81.8	1.356
C2 (RoPE)	96.9	68.0	0.007	95.3	1.160
C3 (GQA)	85.9	14.1	0.042	77.2	1.411
C4 (RMSNORM)	89.9	30.1	0.021	85.8	1.303
C5 (BLT)	99.9	88.3	0.002	-	0.893
Baseline A	66.1	0.0	0.832	-	-
Baseline B	67.4	0.0	0.836	-	-

5.2 Efficiency Comparison

Table 2: Efficiency metrics on NVIDIA RTX 3050 Mobile (4GB VRAM).

Config	Params	Epoch (s)	Tok/sec	Peak VRAM (MB)
C1 (Base)	7.89M	98.1	9,461	3,019
C2 (RoPE)	7.89M	109.0	8,520	3,017
C3 (GQA)	6.70M	94.0	9,882	3,004
C4 (RMSNorm)	7.88M	96.6	9,615	2,837
C5 (BLT)	7.77M	115.1	20,996	3,047

5.3 Training Curves

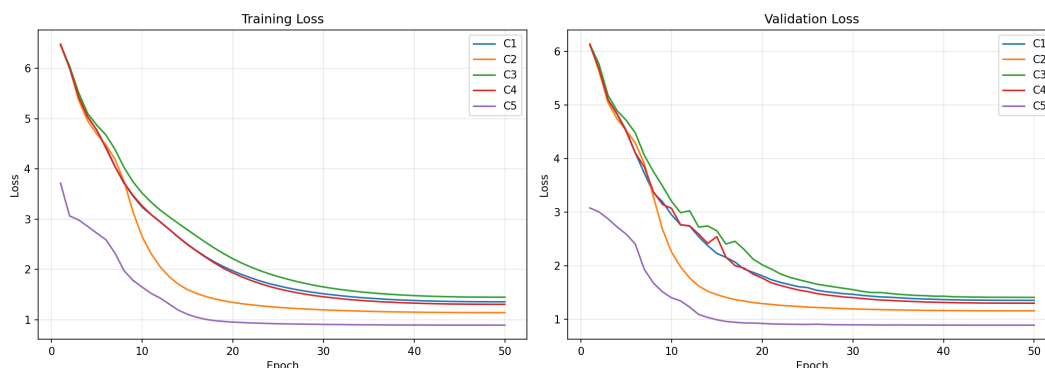


Figure 1: Validation loss over 50 epochs. C5 converges fastest to the lowest loss. C2 is the best subword model. C3 converges slowest.

6 Observations and Analysis

6.1 C2 (RoPE): Best Subword Model

RoPE gave the largest boost among C1–C4: +8.7% bit accuracy, $3\times$ sequence accuracy (21.8% $\rightarrow$ 68.0%). This is due to the fact that the cipher is *position-invariant* (the same 8-bit pattern always decodes to the same character regardless of position). Sinusoidal PE injects absolute position, which is irrelevant noise for this task. RoPE instead encodes relative position in the attention scores, letting the model focus on local byte patterns.

6.2 C3 (GQA)

GQA cut parameters by 15% (7.89M $\rightarrow$ 6.70M) and ran slightly faster (94s vs. 98s/epoch). However, accuracy dropped by -2.3% bit accuracy and -7.7% sequence accuracy. The 4:1 KV sharing ratio was probably too aggressive at this scale the cipher requires the model to maintain distinct attention patterns for different bit positions within each 8-bit group, and sharing KV heads across 4 query heads would limit this capacity.

6.3 C4 (RMSNorm)

RMSNorm improved accuracy by +1.7% bit and +8.3% sequence over the base, with the lowest peak VRAM (2,837 MB, -6%). The improvement comes from the simpler normalization landscape removing the mean subtraction reduces the number of competing gradients during back-propagation

6.4 C5 (BLT)

BLT achieved **99.9%** bit accuracy and **88.3%** sequence accuracy-far beyond any subword model. Two factors explain this:

1. **No tokenization bottleneck:** BPE compresses the cipher into ~ 150 tokens whose boundaries don't align with the 8-bit character structure. BLT operates on the raw 128-byte representation, where each byte is exactly one cipher character.
2. **Compact sequences:** 128 bytes vs. ~ 150 BPE tokens means the attention matrices are smaller, the model trains faster per-token, and there's less room for the model to "get confused" by misaligned boundaries.

Memory tradeoff: For the presets of the parameters taken for both C1 and C5, we actually observed negligent memory tradeoff with both models having a similar Compute requirement in memory (only 28 more MB of VRAM taken by C5).

Speed tradeoff: C5's epoch time is 17% longer (115s vs. 98s) because of the local encoder overhead. However, its throughput in tokens/sec is $2.2\times$ higher because the sequences are shorter, meaning the actual per-batch compute is fast-the overhead is purely from the additional local attention layers.

6.5 Overall Ranking

By bit-level accuracy: **C5 (99.9%) $\gg$ C2 (96.9%) > C4 (89.9%) > C1 (88.2%) > C3 (85.9%)**. We observed that the tokenization strategy helped in the results far more than just the architectural changes

Component	C1 (Base)	C5 (BLT)
Source Embedding	$1000 \times 256 = 256\text{K}$	$259 \times 128 = 33\text{K}$
Target Embedding	$1000 \times 256 = 256\text{K}$	$259 \times 256 = 66\text{K}$
Local Encoder	-	$\sim 330\text{K}$
Global Enc/Dec	$\sim 7.37\text{M}$	$\sim 7.37\text{M}$
Total	7.89M	7.77M